



Update: Insecure Configuration Flag Templates (06/2026)

Sommario

Nuova **famiglia di detection** per il CodeQL Security Agent: **insecure configuration flag** (point-detection, no dataflow). Rileva flag di configurazione insicuri passati come keyword argument a chiamate note (es. `verify=False`, `shell=True`, `autoescape=False`, `debug=True`, hash deboli, randomness non-crypto, cookie senza flag di sicurezza).

È il primo dei nuovi template pianificati (vedi TODO "nuovi template in modo strutturato"). Approccio: **7 tool dedicati** (nomi che il modello locale riconosce facilmente) + **1 tool generico parametrizzato** per i pattern non coperti.

Pattern coperti

Tool dedicato	CWE	Pattern rilevato
<code>check_insecure_verify_false</code>	CWE-295	<code>requests.get/post/...</code> (<code>verify=False</code>)
<code>check_insecure_shell_true</code>	CWE-78	<code>subprocess.run/Popen/...</code> (<code>shell=True</code>)
<code>check_insecure_autoescape_false</code>	CWE-79	<code>jinja2.Environment/Template</code> , <code>flask.render_template_string</code> con <code>autoescape=False</code>
<code>check_insecure_debug_true</code>	CWE-489	<code>Flask(..., debug=True)</code>
<code>check_insecure_weak_hash</code>	CWE-327	<code>hashlib.md5()/sha1()</code>
<code>check_insecure_weak_randomness</code>	CWE-330	<code>random.random()/randint()/...</code> per uso crypto
<code>check_insecure_cookie_flags</code>	CWE-614	<code>set_cookie(secure=False /</code> <code>httponly=False / samesite='None')</code>

Tool generico: `run_insecure_config_flag_query(db_path, module, functions, param_name, insecure_value, cwe)` — renderizza a runtime un template parametrizzato per coprire pattern custom (gestisce sia valori booleani sia stringhe, con validazione regex degli identificatori).

Come è costruito (pattern CodeQL)

Ispirato alla query ufficiale `Jinja2WithoutEscaping.ql` del checkout `vscode-codeql-starter`. Cuore del rilevamento:

```
import python
import semmle.python.ApiGraphs

from API::CallNode call
where
  call = API::moduleImport("<module>").getMember(["<fn>",
  ...]).getACall() and
  call.getKeywordParameter("<param>")
    .getAValueReachingSink()
    .asExpr()
    .(ImmutableLiteral)
    .booleanValue() = <insecure>
select call, "<message>"
```

Tutti i template usano i placeholder `{{CWE_ID_SUFFIX}}` / `{{CWE_TAG_LINE}}` per **incidere il CWE nei @tags**, così il finding porta il CWE come provenienza deterministica (non un'ipotesi del modello) — coerente con il grounding già adottato per taint/api-misuse.

Integrazione nel sistema

- `query_templates/` — 7 template specifici `.ql.tpl` + 1 generico (`insecure_config_flag_generic.ql.tpl`).
- `codeql_mcp_server.py` — `INSECURE_CONFIG_FLAG_TEMPLATES` (registry) + `_make_insecure_config_tool()` genera i 7 `check_insecure_*` in automatico;

`run_insecure_config_flag_query` per il generico. `list_cwes()` ora instrada `detection="insecure_config_flag"` al tool dedicato (via `actions.tool`).

- `knowledge/cwe_wiki.json` — 7 voci con `detection: "insecure_config_flag"`, `actions.tool`, `candidate_patterns`, `how_to_verify`.
 - `build_actions.py` — nuovo ramo `_insecure_config_flag_actions()` + mappa `INSECURE_CONFIG_TOOLS`.
 - `agent.py` — `SYSTEM_PROMPT` aggiornato: il modello ora cerca esplicitamente gli insecure configuration flags durante la lettura del codice.
-

Verifica

- **Compilazione:** tutti i template compilano con `codeql query compile` (0 errori).
 - **End-to-end su repo di prova** (`_insecure_config_testrepo`, contiene un'istanza di ogni pattern): **7/7 tool dedicati** trovano esattamente 1 finding ciascuno con CWE corretto; il tool generico rileva `verify=False` (CWE-295).
 - **Test automatici** (`tests_agent.py`): aggiunti test unit (routing `list_cwes`, registrazione dei 7 tool, validazione input del generico), test di compilazione parametrizzati sui 7 template (marker `codeql`) e test di integrazione (generico + negativo su repo pulito). Suite veloce: **29/29 pass**.
-

Note

- Detection **point** (no dataflow): un finding flagga la configurazione; severità da confermare col contesto (`read_file_snippet`) — es. `debug=True` dipende dall'ambiente di deploy.
- Estendibile: aggiungere una voce a `INSECURE_CONFIG_FLAG_TEMPLATES` + il `.ql.tmpl` → il tool `check_insecure_<key>` nasce in automatico.
- README di progetto aggiornato di conseguenza (tool list, naming automatico del report, sezione "estendere un template", comandi test).